

DSAA3073: Theories in Data Science

Week 11: Deep Learning II: Neural Tangent Kernel & Implicit Regularization

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-07-09

Explorative projection

Background: Reproducing Kernel Hilbert Spaces

Looking Ahead

Why review kernel methods now?

In this section, we review RKHS theory from Weeks 6-8. This mathematical framework will become essential for understanding the Neural Tangent Kernel—it turns out that infinitely wide neural networks behave exactly like kernel machines, and RKHS theory is the language we need to make this connection precise.

Before diving into Neural Tangent Kernel theory, we need essential background on kernel methods. This section provides a self-contained review—everything you need to connect deep learning to kernel theory.

Kernels as Similarity Measures

Definition: Kernel Function

A **kernel** is a function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ that measures similarity between inputs. For a kernel to be valid, it must be:

1. **Symmetric:** $k(x, x') = k(x', x)$
2. **Positive semi-definite:** For any $x_1, \dots, x_n$ and $c_1, \dots, c_n$:

$$\sum_{i,j} c_i c_j k(x_i, x_j) \geq 0 \quad (1)$$

Key examples:

- Linear kernel: $k(x, x') = \langle x, x' \rangle$
- Polynomial kernel: $k(x, x') = (\langle x, x' \rangle + c)^d$
- RBF (Gaussian) kernel: $k(x, x') = \exp(-\gamma \|x - x'\|^2)$

Intuition: Kernels implicitly map inputs to a high-dimensional feature space where linear methods work well, without ever computing the mapping explicitly.

Key Takeaway

Why kernels matter for neural networks

Neural networks define **implicit kernels** through their architecture and initialization. Understanding kernel theory helps us:

- Analyze what functions neural networks can represent
- Understand training dynamics in the infinite-width limit
- Connect deep learning to classical machine learning theory

The key bridge: In the infinite-width limit, neural networks become kernel machines with a specific kernel (the NTK) determined by their architecture.

Reproducing Kernel Hilbert Spaces

Definition: Reproducing Kernel Hilbert Space (RKHS)

A **Reproducing Kernel Hilbert Space** $\mathcal{H}$ associated with kernel k is a Hilbert space of functions $f : \mathcal{X} \rightarrow \mathbb{R}$ with the **reproducing property**:

$$f(x) = \langle f, k(x, \cdot) \rangle_{\mathcal{H}} \quad \text{for all } f \in \mathcal{H}, x \in \mathcal{X} \quad (2)$$

where $k(x, \cdot)$ is the function $x' \mapsto k(x, x')$.

What this means: Evaluating a function f at point x is just an inner product in the RKHS! This seemingly abstract property has profound consequences.

Key properties:

1. Every positive semi-definite kernel uniquely defines an RKHS
2. Functions in the RKHS have a norm: $\|f\|_{\mathcal{H}}^2$
3. The **representer theorem**: solutions to regularized learning problems have form $f(x) = \sum_{i=1}^n \alpha_i k(x, x_i)$

Looking Ahead

Connection to Neural Tangent Kernel

Why do we care about RKHS norms and the representer theorem?

When we analyze infinitely wide neural networks, their predictions will live in the RKHS defined by the Neural Tangent Kernel. The RKHS norm $\|f\|_{\mathcal{H}}^2$ will control generalization (just like it does for kernel ridge regression), and the representer theorem will tell us that the learned function has a specific form: a weighted combination of kernel evaluations at training points.

This gives us a complete theory for analyzing neural networks—if they stay in the lazy training regime where the kernel doesn't change.

Kernel Ridge Regression

Example: Learning in RKHS

Ridge regression in an RKHS:

$$\min_{f \in \mathcal{H}} \left(\sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}}^2 \right) \quad (3)$$

Solution: $f^*(x) = \sum_{i=1}^n \alpha_i k(x, x_i)$ where $\alpha = (K + \lambda I)^{-1} y$

Here K is the **kernel matrix**: $K_{ij} = k(x_i, x_j)$.

Why this works: The RKHS norm $\|f\|_{\mathcal{H}}^2$ penalizes function complexity. Kernel ridge regression balances fit (training error) against smoothness (RKHS norm)—the same bias-variance tradeoff from Week 4, now in function space.

Optimization and generalization:

- **Optimization:** Kernel ridge regression is a convex problem with a closed-form solution
- **Generalization:** RKHS norm bounds give sample complexity $O(\frac{1}{\epsilon^2})$ (Week 7)
- **Limitation:** Fixed feature space—no adaptation to the task

Key Takeaway

The kernel ridge regression template

The key mathematical pattern:

1. Start with a kernel $k(x, x')$ that defines similarity
2. This kernel induces an RKHS $\mathcal{H}$ of functions
3. Learning minimizes: training error + RKHS norm penalty
4. Solution has form $f(x) = \sum_{i=1}^n \alpha_i k(x, x_i)$ (representer theorem)
5. Generalization controlled by RKHS norm $\|f\|_{\mathcal{H}}^2$

Neural Tangent Kernel follows this exact pattern when networks are infinitely wide!

The Feature Space Perspective

🧠 Mental Model: Kernels as Implicit Feature Maps

Two equivalent views of kernel methods:

View 1: Implicit feature space (kernel trick)

- Work directly with kernel $k(x, x')$
- Never compute feature map $\varphi(x)$ explicitly
- Efficient when feature space is high-dimensional or infinite

View 2: Explicit feature space

- Kernel $k(x, x') = \langle \varphi(x), \varphi(x') \rangle$ for some feature map φ
- Learn linear model in feature space: $f(x) = \langle w, \varphi(x) \rangle$
- Useful for understanding and computation

For neural networks: The NTK uses View 2! The feature map is $\varphi(x) = \nabla_{\theta} f(x; \theta_0)$ (parameter gradients at initialization). Training a wide neural network is equivalent to linear regression on these features.

Example: Linear kernel

- Kernel: $k(x, x') = \langle x, x' \rangle$
- Feature map: $\varphi(x) = x$ (identity)
- RKHS: Linear functions $f(x) = \langle w, x \rangle$

Example: Polynomial kernel

- Kernel: $k(x, x') = (\langle x, x' \rangle + 1)^2$
- Feature map (for $d = 2$ dimensions): $\varphi(x) = (x_1^2, \sqrt{2}x_1x_2, x_2^2, \sqrt{2}x_1, \sqrt{2}x_2, 1)$
- RKHS: Degree-2 polynomials

Example: Neural Tangent Kernel

- Kernel: $\Theta(x, x') = \langle \nabla_{\theta} f(x; \theta_0), \nabla_{\theta} f(x'; \theta_0) \rangle$
- Feature map: $\varphi(x) = \nabla_{\theta} f(x; \theta_0)$ (parameter gradients at initialization)
- RKHS: Functions reachable by linear combinations of gradient features

🧐 Looking Ahead

The key question for Section 2

Neural networks start with **nonlinear** training dynamics: $f(x; \theta_t)$ depends on parameters θ_t in a highly nonlinear way (composition of layers, activations, etc.).

How do we get from this nonlinear optimization problem to the linear kernel ridge regression framework we just reviewed?

The answer: **Linearization around initialization**. When networks are wide enough, parameters barely move during training. This lets us approximate the nonlinear network by its first-order Taylor expansion, reducing training to kernel regression with the Neural Tangent Kernel.

RKHS Theory: Key Results

We summarize the essential theoretical tools from Weeks 6-8 that we'll need for NTK analysis.

Theorem: Representer Theorem

Setup: Minimize regularized risk in an RKHS:

$$\min_{f \in \mathcal{H}} \left[\sum_{i=1}^n \ell(y_i, f(x_i)) + \lambda \|f\|_{\mathcal{H}}^2 \right] \quad (4)$$

where ℓ is any loss function and $\lambda > 0$ is the regularization parameter.

Conclusion: The optimal solution has the form:

$$f^*(x) = \sum_{i=1}^n \alpha_i k(x, x_i) \quad (5)$$

for some coefficients $\alpha_1, \dots, \alpha_n \in \mathbb{R}$.

Interpretation: Despite optimizing over an infinite-dimensional function space $\mathcal{H}$, the solution lives in the finite-dimensional span of kernel evaluations at training points. This makes the problem tractable!

Why this matters for neural networks: When an infinitely wide network is trained with squared loss and ridge regularization, the representer theorem guarantees the learned function is a weighted combination of NTK evaluations at training points.

Theorem: RKHS Norm Generalization Bound

Setup: Learn function $f \in \mathcal{H}$ by minimizing training error plus RKHS norm penalty:

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \left[\frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}}^2 \right] \quad (6)$$

Generalization bound: With probability $\geq 1 - \delta$:

$$|R(\hat{f}) - \hat{R}_S(\hat{f})| \leq O \left(\sqrt{\frac{\|f^*\|_{\mathcal{H}}^2 + \sigma^2}{\lambda n}} + \sqrt{\frac{\log(\frac{1}{\delta})}{n}} \right) \quad (7)$$

where f^* is the true function and σ^2 is the noise variance.

Key insight: Generalization controlled by the RKHS norm $\|f^*\|_{\mathcal{H}}^2$ of the target function. If the target has small RKHS norm (smooth in the kernel's feature space), kernel methods generalize well.

Implication for NTK: Generalization depends on whether the target function has small norm in the NTK's RKHS. If the target requires feature learning (hierarchical representations), the NTK RKHS norm may be large, leading to poor generalization despite good optimization.

🌟 Key Takeaway

The RKHS generalization paradigm

Three key quantities control learning in RKHS:

1. **Kernel eigenspectrum:** Determines optimization speed (larger eigenvalues $\rightarrow$ faster convergence)
2. RKHS norm $\|f^*\|_{\mathcal{H}}^2$: Controls sample complexity (smaller norm $\rightarrow$ better generalization)
3. **Effective dimension:** Number of significant kernel eigenvalues (lower dimension $\rightarrow$ fewer samples needed)

For neural networks: These three quantities are determined by the architecture and initialization (which define the NTK), not by learning. This explains why NTK theory works for optimization but struggles with generalization—it can't capture how networks adapt their representations to make the target function “simpler.”

Kernel Methods vs. Feature Learning: A Preview

🧐 Looking Ahead

The central tension in Week 11

Kernel methods (including NTK in the infinite-width limit):

- **Strengths:** Convex optimization, theoretical guarantees, closed-form solutions
- **Limitations:** Fixed feature space determined by kernel, cannot adapt to task structure

Feature learning (finite-width neural networks):

- **Strengths:** Learns task-specific representations, exploits hierarchical structure
- **Limitations:** Nonconvex optimization, fewer theoretical guarantees

The key question: When does the infinite-width limit (kernel regime) explain real neural networks, and when does feature learning dominate?

This tension will drive Section 2 (NTK theory) and Section 3 (lazy training vs. feature learning).

A concrete comparison:

Property	Kernel Methods (NTK)	Feature Learning
Feature space	Fixed at initialization	Adapts during training
Optimization	Convex (linear in features)	Nonconvex
Theory	Complete (convergence + generalization)	Partial (implicit regularization)

Sample complexity	Depends on RKHS norm of target	Can adapt to task structure
Architecture matters?	Only through initial kernel	Strongly affects learned features
Best regime	Smooth targets matching init kernel	Hierarchical/compositional structure

🎯 Key Takeaway

What we'll discover in Section 2

The Neural Tangent Kernel theory shows:

1. Infinitely wide networks are equivalent to kernel machines
2. Training dynamics become **linear** (gradient flow in RKHS)
3. Convergence is guaranteed when NTK is non-degenerate

But this comes with a cost:

- Networks in the NTK regime cannot learn features
- They're stuck with representations from random initialization
- Finite-width networks that learn features often generalize better

The resolution: NTK explains **one regime** (lazy training), but real deep learning often operates in the **feature learning regime** where theory is less complete but performance is better.

Summary: RKHS Background

Core concepts reviewed:

1. **Kernels** measure similarity: $k(x, x') = \langle \varphi(x), \varphi(x') \rangle$ for implicit feature map φ
2. **RKHS** is the space of functions with reproducing property: $f(x) = \langle f, k(x, \cdot) \rangle_{\mathcal{H}}$
3. **Kernel ridge regression** minimizes training error + RKHS norm penalty
4. **Representer theorem** says solutions live in span of kernel evaluations at training points
5. **Generalization** controlled by RKHS norm of target function

Connection to neural networks:

- Wide networks define implicit kernels through their architecture (the NTK)
- In the infinite-width limit, training becomes kernel ridge regression
- The NTK feature map is $\varphi(x) = \nabla_{\theta} f(x; \theta_0)$ (gradients at initialization)
- This explains optimization but misses feature learning

 Looking Ahead**Next: Neural Tangent Kernel Theory**

Now that we have the RKHS machinery in place, we can tackle the main questions:

1. **How do we derive the NTK?** What is the kernel for a given architecture?
2. **When does the kernel stay constant?** What width is needed for lazy training?
3. **What are the training dynamics?** How fast does the network converge?
4. **When does NTK theory break down?** When do real networks learn features instead?

Section 2 will answer these questions, revealing both the power and limitations of kernel methods for understanding deep learning.

Neural Tangent Kernel Theory

This section connects neural networks to kernel methods from Weeks 6-8. The key insight: very wide neural networks behave almost like kernel methods.

Opening Challenge

For very wide networks trained with small learning rates, training dynamics are almost linear! This surprising connection links neural networks to kernel methods, providing theoretical guarantees for optimization.

Connection

Connection to Week 11 Background Section (RKHS): Recall that kernels measure similarity $k(x, x')$ between inputs and define a **reproducing kernel Hilbert space (RKHS)** where learning takes place. The Neural Tangent Kernel plays the same role for infinite-width networks, connecting deep learning to kernel theory.

Linearization and the NTK

Definition: Neural Tangent Kernel

The **Neural Tangent Kernel (NTK)** at initialization θ_0 is:

$$\Theta(x, x') = \nabla_{\theta} f(x; \theta_0)^T \nabla_{\theta} f(x'; \theta_0) = \sum_{i=1}^p \frac{\partial f(x; \theta_0)}{\partial \theta_i} \frac{\partial f(x'; \theta_0)}{\partial \theta_i} \quad (8)$$

This measures the alignment between parameter gradients at two inputs x and x' . Intuitively, if $\Theta(x, x')$ is large, changing parameters to fit x will also change the output at x' (the two inputs are similar from the network's perspective).

Example: NTK for a Linear Model

For the ordinary linear model $f_w(x) = w^T x$, the parameter gradient is $\nabla_w f_w(x) = x$. Therefore the NTK is

$$\Theta(x, x') = x^T x', \quad (9)$$

exactly the standard linear kernel. This is why parameter gradients can be viewed as features.

Example: NTK for Two-Layer Network

For a two-layer network $f(x) = \left(\frac{1}{\sqrt{m}}\right) \sum_{i=1}^m a_i \sigma(w_i^T x)$ with $a_i \in \{-1, +1\}$ fixed (only training the input weights w_i):

$$\Theta(x, x') = \frac{1}{m} \sum_{i=1}^m \sigma'(w_i^T x) \sigma'(w_i^T x') x^T x' \quad (10)$$

where σ' is the derivative of the activation function.

As width $m \rightarrow \infty$, by the **law of large numbers** (the average of many independent random variables converges to their expectation):

$$\Theta(x, x') \rightarrow \mathbb{E}_{w \sim \mathcal{N}(0, I)} [\sigma'(w^T x) \sigma'(w^T x')] x^T x' \quad (11)$$

The NTK becomes deterministic! The randomness from initialization averages out.

Note: Because a_i are fixed, only the input-weight gradients contribute. If output weights are also trained, the standard two-layer NTK has an additional covariance term from the a_i gradients.

Pause and Think

Question: Why does the NTK become deterministic as width increases?

Hint: Think about the law of large numbers. What happens to the average of many i.i.d. (independent and identically distributed) random variables?

Answer:

The NTK is an average over m neurons: $\Theta = \left(\frac{1}{m}\right) \sum_{i=1}^m$ contribution from neuron i . Each neuron is initialized independently with random weights. As $m \rightarrow \infty$, this average concentrates around its expectation by the law of large numbers (the sample average converges to the population mean), becoming deterministic. The randomness from initialization averages out.

Example: Computing NTK for ReLU

For ReLU activation $\sigma(z) = \max(0, z)$, the derivative is $\sigma'(z) = \mathbb{1}_{z>0}$ (the indicator function: 1 if $z > 0$, and 0 otherwise) away from $z = 0$. The value at $z = 0$ is irrelevant under continuous Gaussian initialization because it occurs with probability zero.

The NTK becomes:

$$\Theta(x, x') = \mathbb{E}_{w \sim \mathcal{N}(0, I)} [\mathbb{1}_{w^T x > 0} \mathbb{1}_{w^T x' > 0}] x^T x' \quad (12)$$

The expectation is the probability that both $w^T x$ and $w^T x'$ are positive (both projections land in the positive half-space):

$$\mathbb{E}[\mathbb{1}_{w^T x > 0} \mathbb{1}_{w^T x' > 0}] = \mathbb{P}(w^T x > 0 \text{ and } w^T x' > 0) \quad (13)$$

For Gaussian w , this depends on the angle θ between x and x' . Using geometric arguments from probability theory:

$$\mathbb{P}(w^T x > 0 \text{ and } w^T x' > 0) = \frac{\pi - \theta}{2\pi} = \frac{1}{2} - \frac{\theta}{2\pi} \quad (14)$$

where $\cos \theta = \frac{x^T x'}{\|x\|_2 \|x'\|_2}$ is the normalized dot product.

Therefore, the ReLU NTK has a closed form:

$$\Theta(x, x') = \left(\frac{\pi - \theta}{2\pi} \right) x^T x' = \left(\frac{1}{2} - \frac{\arccos\left(\frac{x^T x'}{\|x\|_2 \|x'\|_2}\right)}{2\pi} \right) x^T x' \quad (15)$$

This is a specific kernel determined entirely by the ReLU activation and Gaussian initialization—no learning involved!

🧠 Mental Model: NTK as Feature Learning Frozen at Initialization

Understanding the NTK regime:

Standard neural network training:

- Parameters θ change significantly during training
- Features $\varphi(x) = \nabla_{\theta} f(x; \theta)$ evolve (feature learning)
- Network output $f(x; \theta)$ is highly nonlinear in parameters

NTK regime (infinite width):

- Parameters change infinitesimally (each neuron's contribution vanishes as $\frac{1}{\sqrt{m}}$)
- Features $\varphi(x) \approx \nabla_{\theta} f(x; \theta_0)$ stay nearly constant
- Training becomes **linear** in features frozen at initialization
- Equivalent to kernel regression with kernel $\Theta(x, x') = \varphi(x)^T \varphi(x')$

Key insight: Infinite width “freezes” the features at random initialization. Training only adjusts coefficients, not features. This is why NTK connects to kernel methods (Week 11 Background Section)!

Limitation: Real networks (finite width) learn features. NTK explains optimization but misses representation learning.

Theorem: NTK Training Dynamics (Jacot et al. 2018; simplified)

Assume:

- **Architecture:** Fully-connected network with L layers and width m per hidden layer
- **Activation:** $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is Lipschitz continuous and bounded (e.g., tanh, sigmoid, or smoothed ReLU)
- **Initialization:** Weights $\theta_0 \sim \mathcal{N}(0, \sigma_w^2 I)$ with appropriate scaling (e.g., $\sigma_w = \frac{1}{\sqrt{m}}$)
- **Loss:** Squared loss $\mathcal{L}_{S(\theta)} = \frac{1}{2n} \sum_{i=1}^n (f(x_i; \theta) - y_i)^2$ on training set $S = \{(x_1, y_1), \dots, (x_n, y_n)\}$
- **Training:** Gradient flow (continuous-time gradient descent) $\frac{d\theta}{dt} = -\nabla_{\theta} \mathcal{L}_{S(\theta_t)}$
- **Regime:** Infinite-width limit $m \rightarrow \infty$ with fixed dataset size n

Then: The network output dynamics become linear:

$$\frac{d\mathbf{f}_t}{dt} = -\frac{1}{n} \Theta(\mathbf{f}_t - \mathbf{y}) \quad (16)$$

where:

- $\mathbf{f}_t = (f(x_1; \theta_t), \dots, f(x_n; \theta_t))^T \in \mathbb{R}^n$ = network outputs on training data
- $\mathbf{y} = (y_1, \dots, y_n)^T \in \mathbb{R}^n$ = target values
- $\Theta \in \mathbb{R}^{n \times n}$ = NTK Gram matrix with $\Theta_{ij} = \Theta(x_i, x_j)$
- $\Theta(x, x') = \text{NTK kernel (constant in infinite-width limit)}$

This is a **linear** ODE, unlike the original nonlinear neural network dynamics.

Proof. Sketch of the key steps:

1. **Gradient descent update:**

$$\frac{d\theta_t}{dt} = -\nabla_{\theta} \mathcal{L}_{S(\theta_t)} = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; \theta_t)) \nabla_{\theta} f(x_i; \theta_t)$$

2. Network output evolution:

$$\begin{aligned} \frac{df(x; \theta_t)}{dt} &= \nabla_{\theta} f(x; \theta_t)^T \frac{d\theta_t}{dt} \\ &= \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; \theta_t)) \nabla_{\theta} f(x; \theta_t)^T \nabla_{\theta} f(x_i; \theta_t) \\ &= \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; \theta_t)) \Theta_t(x, x_i) \end{aligned} \quad (18)$$

When is Θ positive definite?

For non-degenerate data (no duplicates, sufficient diversity), the NTK Gram matrix is positive definite with high probability under random initialization. The minimum eigenvalue $\lambda_{\min}$ depends on data geometry and kernel properties.

3. **Infinite-width limit:** As width $m \rightarrow \infty$, on a fixed finite dataset and over suitable training horizons, the NTK remains approximately constant: $\Theta_t(x, x') \approx \Theta_0(x, x') = \Theta(x, x')$.
4. This gives the linear dynamics: $\frac{df_t}{dt} = -\frac{1}{n} \Theta(f_t - y)$.

■

Theorem: Exponential Convergence in NTK Regime

Assume:

- NTK training dynamics (from previous theorem): $\frac{df_t}{dt} = -\frac{1}{n} \Theta(f_t - y)$
- **Non-degenerate NTK:** Gram matrix $\Theta \in \mathbb{R}^{n \times n}$ is positive definite with smallest eigenvalue $\lambda_{\min}(\Theta) > 0$
- **Data condition:** Training inputs $\{x_1, \dots, x_n\}$ are distinct and non-degenerate (ensures positive definiteness)

Then: The training error decays exponentially:

$$\|f_t - y\|_2 \leq \exp\left(-\frac{\lambda_{\min} t}{n}\right) \|f_0 - y\|_2 \quad (19)$$

Interpretation: Convergence rate depends on $\frac{\lambda_{\min}}{n}$. Larger minimum eigenvalue $\rightarrow$ faster convergence. Training achieves ε -accuracy in $O\left(\frac{n}{\lambda_{\min}} \log\left(\frac{1}{\varepsilon}\right)\right)$ time.

Proof. The linear ODE $\frac{df_t}{dt} = -\frac{1}{n} \Theta(f_t - y)$ has solution:

$$f_t - y = \exp\left(-\frac{\Theta t}{n}\right) (f_0 - y) \quad (20)$$

Since Θ is positive definite:

$$\|f_t - y\|_2 \leq \left\| \exp\left(-\frac{\Theta t}{n}\right) \right\| \|f_0 - y\|_2 \leq \exp\left(-\frac{\lambda_{\min} t}{n}\right) \|f_0 - y\|_2 \quad (21)$$

■

Key Takeaway

In the infinite-width limit, neural network training behaves like **kernel gradient flow** or **kernel interpolation** in the NTK RKHS (reproducing kernel Hilbert space associated with kernel Θ); with explicit ridge regularization, the predictor corresponds to kernel ridge regression. Dynamics are linear under the constant-NTK approximation and converge exponentially when the empirical NTK Gram matrix Θ is positive definite (all eigenvalues positive).

Connection

Week 6-8 connection: The NTK plays the role of kernel $k(x, x')$ in kernel gradient flow/interpolation (the dynamics of continuously optimizing in the RKHS); with ridge regularization (adding $\lambda \|f\|_{\mathcal{H}}^2$ penalty), it connects to kernel ridge regression (introduced in Week 7). But unlike hand-designed kernels (polynomial, RBF), the NTK is determined by architecture and initialization—it's not chosen a priori but emerges from the network structure.

Sample complexity implications: In the NTK regime, sample complexity matches kernel methods: $\tilde{O}(\frac{1}{\varepsilon^2})$ samples for ε -accuracy under RKHS norm bounds (Week 7). This can be suboptimal compared to feature learning when the target function has hierarchical structure exploitable by deep representations—highlighting the limitation of lazy training.

✓ Checkpoint

NTK theory summary:

1. Infinite-width networks behave like kernel methods with kernel $\Theta(x, x')$ determined by architecture and initialization
2. Training dynamics become **linear** (gradient flow in RKHS, unlike the original non-linear neural network dynamics)
3. Convergence is exponential when the NTK Gram matrix is positive definite (like kernel ridge regression from Week 7)
4. NTK is determined by architecture and initialization, not learned during training (features are frozen at initialization)

Key limitation: NTK assumes features don't change during training (lazy training regime). Real finite-width networks learn features—this is their main advantage over kernel methods!

Lazy Training vs. Feature Learning

Definition: Training Regimes

Neural networks exhibit different behaviors depending on width:

- **Lazy training:** NTK stays approximately constant during training. The network behaves like a kernel method—features are frozen.
- **Feature learning:** NTK changes significantly during training. The network learns task-specific representations—features evolve during training.

Width requirement impractical

Original proof requires $m = \Omega(n^6)$ neurons—impractical! Recent work improves to $m = O(n^2)$ or $m = O(n)$ under stronger assumptions. The theorem shows convergence is possible in theory, not that it's necessary in practice.

Theorem: Global Convergence for Overparameterized Networks (Du et al. 2019; simplified)

Assume:

- **Architecture:** Two-layer network $f(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^T x)$ with m hidden units
- **Width:** $m \geq C \frac{n^6}{\delta^2 \lambda_0^2}$ for constant C (extremely overparameterized)
- **Initialization:** Weights $w_i \sim \mathcal{N}(0, I)$, second-layer weights $a_i \in \{-1, +1\}$ fixed
- **Activation:** ReLU or smooth activation with bounded derivatives
- **Data:** Training set $S = \{(x_1, y_1), \dots, (x_n, y_n)\}$ with $\|x_i\|_2 = 1$ (normalized)
- **Non-degeneracy:** Minimum eigenvalue $\lambda_0 = \lambda_{\min}(\Theta_0) > 0$ at initialization
- **Learning rate:** $\eta = O\left(\frac{\lambda_0}{n}\right)$ (sufficiently small)

Then: With probability $\geq 1 - \delta$ over random initialization, gradient descent converges to training loss $\mathcal{L}_{S(\theta_T)} < \varepsilon$ in $T = O\left(\frac{n}{\lambda_0} \log\left(\frac{1}{\varepsilon}\right)\right)$ iterations.

Key insight: Sufficient width ensures NTK stays nearly constant (lazy training), reducing to linear dynamics with guaranteed convergence.

Wrong Path: NTK Theory Explains All of Deep Learning

Why incomplete:

1. Assumes lazy training (networks stay near initialization), but practical networks learn features
2. Requires impractically large width
3. Explains optimization but not generalization
4. Treats all architectures similarly, but architecture matters enormously in practice
5. Ignores the power of feature learning—the main advantage of deep learning over kernels!

What NTK does explain:

- Why very wide networks optimize easily (lazy regime)
- Connection between neural networks and kernel methods
- Lower bound on what networks can achieve (kernels $\subseteq$ neural networks)

What NTK misses:

- Why finite-width networks generalize better than kernels
- How networks learn hierarchical representations
- Why architecture design matters

Connection

Preview of Week 12: NTK explains why networks **optimize** (lazy training $\rightarrow$ kernel methods $\rightarrow$ PL condition $\rightarrow$ exponential convergence). But why do they **generalize**? Week 12 covers implicit regularization, compression, and modern phenomena (grokking, scaling laws).

Mental Model: The Width Spectrum

Understanding training across different widths:

Narrow networks ($m = O(1)$):

- Hard to optimize (many bad local minima)
- If optimization succeeds: good generalization (strong implicit bias)
- High variance (sensitive to initialization)

Moderate width ($m = O(n)$ to $O(n^2)$):

- Easier optimization (fewer bad local minima)
- Feature learning (NTK changes during training)
- Best generalization (sweet spot)
- This is the practical regime!

Infinite width ($m \rightarrow \infty$):

- Easiest optimization (linear dynamics, exponential convergence)
- No feature learning (lazy training)
- Kernel method behavior
- Explains optimization but misses generalization benefits

Key insight: Finite width is crucial for both tractable optimization AND good generalization through feature learning!

This leads us to the next question: NTK theory explains optimization convergence, but we still face the generalization puzzle. In the overparameterized regime, there are infinitely many solutions that perfectly fit the training data—why does gradient descent find one that generalizes well? The answer lies in **implicit regularization**: gradient descent has built-in biases that favor simple, generalizable solutions even without explicit penalties. This is perhaps the deepest insight in modern deep learning theory.

Implicit Regularization in Deep Learning

Modern deep learning presents a puzzle that confounds classical theory: neural networks with millions of parameters trained on thousands of samples achieve zero training error yet generalize well. Classical learning theory predicts catastrophic overfitting in this regime. What prevents it?

🧐 Student Question

“With $p \gg n$ parameters and n training samples, infinitely many solutions achieve zero training loss. Why doesn’t gradient descent overfit? More critically: **which** zero-loss solution does it find among the infinitely many possibilities?”

The Natural Approach: GD Finds Arbitrary Interpolating Solutions

When faced with an overparameterized problem ($p > n$), it seems obvious that gradient descent would find different solutions from different initializations. After all, there are infinitely many parameter configurations that perfectly fit the training data.

The intuitive reasoning:

Consider a simple matrix completion problem: recover a matrix $M^* \in \mathbb{R}^{10 \times 10}$ from only $n = 30$ random measurements. We parameterize $M = UV^T$ where $U, V \in \mathbb{R}^{10 \times 5}$, giving us $p = 100$ parameters for just 30 measurements.

Any factorization satisfying $UV^T = M^*$ on the measured entries achieves zero training loss. There are infinitely many such factorizations (we can apply any rotation: if UV^T works, so does $UR(R^TV^T)$ for any orthogonal R).

The natural prediction:

Running gradient descent from 100 different random initializations should give 100 wildly different solutions:

- Different Frobenius norms: $\|M\|_F \in [5, 50]$
- Different nuclear norms: $\|M\|_* \in [4, 100]$
- Different test errors: highly variable generalization

The optimization landscape has infinitely many global minima (all achieve zero training loss), so which one gradient descent finds should depend sensitively on initialization.

Where This Breaks: The Norm Concentration Crisis

Let’s test this prediction with a concrete experiment.

⚠ Crisis

The Fatal Flaw: Mysterious Consistency

Matrix completion experiment (Gunasekar et al. 2017):

- Task: Recover rank-2 matrix $M^* \in \mathbb{R}^{10 \times 10}$ from 30 random measurements
- Parameterization: $M = UV^T$ where $U, V \in \mathbb{R}^{10 \times 5}$ (100 parameters, 30 constraints)
- Initialization: 100 independent random starting points
- Training: Gradient descent until training loss $< 10^{-6}$

Expected: High variance across initializations (different local minima)

Observed: Shockingly consistent results!

Statistic	Frobenius Norm	Nuclear Norm	Test MSE
Mean	8.21	15.23	0.045
Std Dev	0.09	0.08	0.002
Min	8.03	15.09	0.042
Max	8.42	15.38	0.048

All 100 random initializations converge to solutions with nearly **identical** norms and test errors! The standard deviation is less than 1% of the mean.

Detailed results from 10 random runs:

Init	Train Loss	Frobenius Norm	Nuclear Norm	Test MSE
Random 1	10^{-7}	8.19	15.19	0.045
Random 2	9×10^{-7}	8.25	15.25	0.046
Random 3	10^{-7}	8.17	15.20	0.044
Random 4	8×10^{-7}	8.23	15.24	0.045
Random 5	10^{-7}	8.20	15.22	0.045
Random 6	9×10^{-7}	8.22	15.23	0.046
Random 7	10^{-7}	8.18	15.21	0.044
Random 8	8×10^{-7}	8.24	15.26	0.046
Random 9	10^{-7}	8.21	15.23	0.045
Random 10	9×10^{-7}	8.20	15.22	0.045

This is not random wandering through the space of solutions! Something is constraining gradient descent to find essentially the same solution every time.

The critical comparison: What if we explicitly construct a random interpolating solution?

Take any matrix M_{random} that achieves zero training loss on the 30 measurements. We can construct many such matrices by randomly filling in the unmeasured entries, then projecting onto rank-5 matrices.

Method	Train Loss	Frobenius Norm	Nuclear Norm	Test MSE
--------	------------	----------------	--------------	----------

GD (mean of 100)	$< 10^{-6}$	8.21	15.23	0.045
Random interpolator 1	$< 10^{-6}$	47.3	152.4	0.487
Random interpolator 2	$< 10^{-6}$	52.1	163.2	0.521
Random interpolator 3	$< 10^{-6}$	44.8	147.9	0.458
Random interp. mean	$< 10^{-6}$	48.1	154.5	0.489

The gap: Random interpolating solutions give $10\times$ worse test error (MSE 0.489 vs 0.045)!

Both approaches achieve perfect training fit (loss $< 10^{-6}$), yet gradient descent consistently finds solutions that generalize an order of magnitude better. This cannot be coincidence.

The Key Insight: Implicit Bias Toward Minimum Norm

The resolution to this mystery lies in understanding what gradient descent is really optimizing.

💡 Aha Moment

The Resolution: Path-Dependent Selection

Gradient descent on overparameterized models has **implicit bias**: among all solutions with zero training loss, it converges to solutions with **minimum norm** in a space determined by the parameterization.

For matrix completion with factorization $M = UV^T$:

- Gradient descent on $U, V \rightarrow$ minimum **nuclear norm** $\|M\|_* = \sum_i \sigma_i$
- Direct gradient descent on $M \rightarrow$ minimum **Frobenius norm** $\|M\|_F = \sqrt{\sum_{ij} M_{ij}^2}$

The parameterization creates an implicit regularization effect!

Why nuclear norm matters:

The nuclear norm $\|M\|_* = \sum_i \sigma_i$ (sum of singular values) is the tightest convex relaxation of matrix rank. Minimizing nuclear norm encourages low-rank solutions, which capture essential structure without overfitting to noise.

This is why gradient descent generalizes well: it finds the **simplest** (lowest-rank) matrix that fits the data, not an arbitrary interpolator.

Verification experiment:

Let's explicitly minimize nuclear norm and compare to gradient descent:

Method	Optimization	Nuclear Norm	Test MSE	Match?
GD on U, V	Minimize $\ Y - UV^T\ ^2$	15.23	0.045	—
Explicit nuclear	Minimize $\ M\ _*$ s.t. $M_\Omega = Y_\Omega$	15.21	0.044	✓
GD on M directly	Minimize $\ Y - M\ ^2$	8.19	0.189	✗

Gradient descent on the factorized parameterization matches explicit nuclear norm minimization! This is not accidental—it's a consequence of the optimization geometry.

The Formal Framework: Implicit Regularization Theory

Definition: Implicit Regularization

An optimization algorithm exhibits **implicit regularization** if, among all solutions achieving the same training loss, it consistently converges to solutions that minimize some notion of complexity (norm, margin, etc.) without explicit regularization terms.

Formally: Let $S = \{\theta : \mathcal{L}(\theta) = 0\}$ be the set of zero-loss solutions. Gradient descent with initialization θ_0 satisfies:

$$\lim_{t \rightarrow \infty} \theta_t = \arg \min_{\theta \in S} \|\theta\|_{\text{implicit}} \quad (22)$$

where $\|\cdot\|_{\text{implicit}}$ depends on the parameterization and initialization.

Key examples of implicit bias:

Model	Parameterization	Implicit Norm	Interpretation
Linear	Direct w	$\ w\ _2$	Small weights
Linear networks	Factored $W = W_L \cdots W_1$	$\ W\ _*$ (nuclear)	Low rank
Deep ReLU nets	Standard	Path norm	Balanced paths
Logistic regression	Separable data	Max margin	SVM solution

Theorem (Implicit bias in linear networks):

Theorem: Nuclear Norm Bias in Matrix Factorization

Consider matrix completion: recover $M \in \mathbb{R}^{d_1 \times d_2}$ from measurements $Y \in \mathbb{R}^n$.

Parameterize as $M = UV^T$ where $U \in \mathbb{R}^{d_1 \times r}$, $V \in \mathbb{R}^{d_2 \times r}$ with $r \geq \text{rank}(M^*)$.

Initialize with small $\sigma \rightarrow 0$: $U_0, V_0 \sim \mathcal{N}(0, \sigma^2 I)$.

Run gradient flow: $\frac{dU}{dt} = -\nabla_U \mathcal{L}$, $\frac{dV}{dt} = -\nabla_V \mathcal{L}$

Then: As $t \rightarrow \infty$, the trajectory converges to:

$$M_\infty = \arg \min_{M: \mathcal{L}(M)=0} \|M\|_* \quad (23)$$

the minimum nuclear norm solution!

Proof sketch:

1. Gradient flow on U, V induces flow on $M = UV^T$
2. The dynamics favor balanced factorizations: $\|U\| \approx \|V\|$
3. Among balanced factorizations with $UV^T = M$, nuclear norm is minimized when U, V contain the singular vectors of M
4. Small initialization ensures gradient flow follows this balanced path

Factors Affecting Implicit Bias

Implicit regularization is not solely determined by the optimizer—it emerges from the **interaction** of architecture, initialization, and optimization dynamics.

Architecture: Parameterization Shapes the Bias

The same model class parameterized differently exhibits different implicit biases.

Example: Linear regression

Parameterization	Implicit Norm	Test MSE	Bias Type
Direct: $y = w^T x$	$\ w\ _2 = 2.4$	0.18	Small norm
2-layer: $y = W_2 W_1 x$	$\ W_2 W_1\ _* = 1.8$	0.09	Low rank
3-layer: $y = W_3 W_2 W_1 x$	Path norm: 1.2	0.06	Balanced paths

Deeper factorizations lead to more aggressive implicit regularization and better generalization, even though all three parameterizations can represent the same linear functions!

Experiment: MNIST with different architectures

Train to zero training error with SGD (momentum = 0.9, learning rate = 0.01):

Architecture	Train Acc	Test Acc	Sharpness	Frequency Bias
Fully connected	99.7%	97.8%	8.4	0.42
CNN	99.8%	98.9%	4.2	0.28
ResNet-like	99.6%	99.1%	2.7	0.18

Key observation: With **identical optimization** (same SGD, same hyperparameters), different architectures exhibit:

- Different sharpness of minima (ResNet finds flatter minima)
- Different frequency bias (ResNet prefers lower frequencies)
- Different generalization (ResNet: 99.1% vs FC: 97.8%)

Architecture is itself a form of implicit regularization!

Initialization: Scale Shapes the Kernel

Even with fixed architecture, initialization scale dramatically affects implicit bias.

Experiment: 2-layer ReLU network with varying initialization

Architecture: $f(x) = W_2 \sigma(W_1 x)$ with width $m = 1000$

Init Scheme	NTK Condition	Steps to 99%	Test Acc	Regime
Standard: $\sigma = \sqrt{2/d}$	23.7	2400	94.3%	Moderate feature learning
Large W_1 : $\sigma_1 = 10\sqrt{2/d}$	186.4	890	87.2%	Near-lazy (frozen W_1)
Large W_2 : $\sigma_2 = 10\sqrt{2/m}$	19.2	1850	91.8%	Moderate feature learning

Key insight: Initialization scale affects:

1. NTK condition number (larger $\sigma \rightarrow$ worse conditioning)
2. Training speed (larger σ can speed up or slow down learning)
3. Implicit regularization (larger σ pushes toward lazy regime)
4. Test accuracy (moderate initialization often best)

The initialization is not just for numerical stability—it shapes the learning dynamics and implicit bias!

Optimizer: Batch Size and Learning Rate Matter

Different optimization algorithms induce different implicit biases even on the same architecture.

Experiment: SGD batch size effects

Task: Fit noisy sine function with 3-layer ReLU network (width 512)

Method	Batch Size	Train MSE	Test MSE	Improvement
Full-batch GD	$n = 50$	10^{-4}	0.045	Baseline
Mini-batch SGD	10	10^{-4}	0.032	+29%
Small-batch SGD	5	10^{-4}	0.023	+49%
SGD	1	10^{-4}	0.027	+40%
Adam (full batch)	$n = 50$	10^{-4}	0.087	−93% (worse)

Key observation: All methods achieve perfect interpolation (train MSE = 10^{-4}), yet small-batch SGD generalizes **49% better** than full-batch GD!

Why does batch size matter?

- Small batches add noise to gradients
- This noise helps escape sharp minima
- Converges to flatter minima with better generalization
- Implicit bias toward “robust” solutions

Comparison of different optimizers:

Optimizer	Train Loss	Test Error	Sharpness	Implicit Bias
GD (full batch)	10^{-6}	4.5%	12.3	Moderate flatness
SGD (batch=32)	10^{-6}	2.3%	4.1	Strong flatness
Adam	10^{-6}	8.7%	24.7	Sharp minima
AdaGrad	10^{-6}	5.2%	15.8	Adaptive per-feature

SGD with small batches finds the flattest minima and generalizes best, even though all optimizers achieve zero training loss!

Implicit Margin Maximization for Classification

For separable classification problems, gradient descent exhibits a different form of implicit regularization: it maximizes the margin.

🧐 Student Question

“For linearly separable binary classification, there are infinitely many separating hyperplanes. Which one does gradient descent choose?”

The setup:

- Data: $\{(x_i, y_i)\}_{i=1}^n$ with $y_i \in \{-1, +1\}$
- Assume linearly separable: $\exists w^*$ such that $y_i \langle w^*, x_i \rangle > 0$ for all i
- Loss: Logistic loss $\mathcal{L}(w) = \sum_{i=1}^n \log(1 + \exp(-y_i \langle w, x_i \rangle))$

The naive expectation:

Once the training loss drops below some threshold (say 10^{-3}), gradient descent has “converged” and we should stop. The solution is whichever separator achieves this threshold first.

The surprising reality:

⚠️ Crisis

Crisis: Early Stopping Leaves Performance on the Table

Binary classification experiment (100 linearly separable 2D points):

Iteration	Train Loss	$\ w\ $	Margin	Test Error
500	0.47	8.1	0.41	8.0%
1000	0.089	14.2	0.67	4.0%
2000	0.011	24.8	0.89	2.0%
5000	1.3×10^{-4}	51.2	0.97	1.0%
10000	1.8×10^{-6}	89.7	0.993	0.8%

Stopping at iteration 500 (loss = 0.47, “seems converged”) gives test error = 8.0%.

Continuing to iteration 10000 (loss ≈ 0 , much smaller) gives test error = 0.8%.

The gap: Early stopping leaves a **10× performance gap!** Even after training loss becomes tiny, continuing training significantly improves generalization.

What’s happening?

As training continues:

1. Training loss $\rightarrow 0$ (exponentially fast)
2. Weight norm $\|w\| \rightarrow \infty$ (linearly in $\log t$)
3. Direction $w / \|w\|$ converges to the max-margin solution
4. Margin approaches maximum possible

Theorem (Soudry et al. 2018):

Theorem: Implicit Margin Maximization

For separable binary classification with logistic or exponential loss, gradient flow converges in direction to the max-margin (SVM) solution:

$$\lim_{t \rightarrow \infty} \frac{w_t}{\|w_t\|} = \frac{w_{\text{SVM}}}{\|w_{\text{SVM}}\|} \quad (24)$$

where w_{SVM} is the hard-margin SVM solution:

$$w_{\text{SVM}} = \arg \max_{w: y_i \langle w, x_i \rangle \geq 1} \min_i \frac{y_i \langle w, x_i \rangle}{\|w\|} \quad (25)$$

Verification experiment:

Track the angular distance between w_t and w_{SVM} :

Iteration	$\ w_t\ $	$\text{angle}(w_t, w_{\text{SVM}})$	Margin / Max-margin	Test Error
100	2.4	34.2°	18%	18.0%
500	8.1	12.7°	62%	8.0%
1000	14.2	7.3°	81%	4.0%
5000	51.2	2.1°	96%	1.0%
10000	89.7	0.8°	99.3%	0.8%

The direction converges to the SVM solution! Gradient descent is implicitly solving the SVM optimization problem, without us ever specifying a margin objective.

Why does this happen?

For logistic loss on separable data:

- Loss $\approx \exp(-\text{margin} \times \|w\|)$ for correctly classified points
- To minimize loss, gradient descent increases $\|w\|$
- Among all directions that separate the data, the max-margin direction is the “slowest to diverge”
- This geometric balance leads to margin maximization

Practical implication:

For separable classification, train until loss is very small ($< 10^{-6}$), not just “small enough” (< 0.01). The extra iterations increase margin and significantly improve generalization.

Deep Networks: More Complex Implicit Biases

For deep nonlinear networks, implicit regularization becomes more nuanced.

Path norm in deep ReLU networks:

Definition: Path Norm

For a deep network $f(x; \theta)$, the **path norm** is:

$$\|f\|_{\text{path}} = \sum_{\text{paths } p} \prod_{e \in p} |w_e| \quad (26)$$

where the sum is over all paths from input to output, and w_e are the edge weights.

Gradient descent on deep networks implicitly minimizes a notion of path norm, favoring solutions where signal is distributed across many paths rather than concentrated in few paths.

Experiment: Deep network implicit bias

3-layer ReLU network on MNIST, vary initialization scale:

Init Scale	$\ W\ _F$ total	Path Norm	Effective Depth	Test Acc	Notes
Small ($\sigma = 0.01$)	142	8.3	2.8	98.2%	Deep network used
Standard ($\sigma = 1$)	1891	47.2	2.1	97.1%	Moderate depth
Large ($\sigma = 10$)	18673	312.4	1.2	93.8%	Effectively shallow

Small initialization leads to:

- Lower path norm (simpler solutions)
- Higher effective depth (all layers contribute)
- Better generalization

Feature learning vs lazy training revisited:

Regime	Width	NTK Change	Test Acc	Implicit Bias
Lazy training	$m \rightarrow \infty$	$< 1\%$	42%	Fixed kernel at init
Feature learning	$m = O(n)$	$> 20\%$	25%	Adaptive features

In the feature learning regime, the network's implicit bias is toward learning features that are:

- Low-frequency in input space
- Hierarchically structured
- Aligned with task structure

This goes beyond simple norm minimization—the network learns **representations** that generalize.

Comparison: Implicit vs Explicit Regularization

How does implicit regularization compare to explicit penalties?

Property	Implicit (GD)	Explicit ($\lambda \ \theta\ ^2$)	Key Difference
----------	---------------	-------------------------------------	----------------

Achieves zero loss?	Yes	No (unless $\lambda = 0$)	Implicit allows interpolation
Norm bound	Emergent from dynamics	Enforced by penalty	Implicit is path-dependent
Hyperparameter	None (learning rate)	Must tune λ	Implicit is “automatic”
Optimization	Can be easier	Adds curvature	Explicit changes objective
Theory	Harder to analyze	Well-understood	Implicit is newer

Concrete comparison experiment:

Matrix completion: 100 parameters, 30 measurements

Method	Train Loss	Nuclear Norm	Test MSE	Notes
GD (implicit)	$< 10^{-6}$	15.2	0.045	Zero training loss
Explicit $\lambda = 0.1$	0.18	12.4	0.052	Nonzero training loss
Explicit $\lambda = 0.01$	0.04	14.1	0.047	Close to implicit
Explicit $\lambda = 0.001$	$< 10^{-5}$	15.0	0.046	Very close to implicit

As $\lambda \rightarrow 0$, explicit regularization approaches implicit regularization. But implicit regularization achieves this automatically without tuning λ !

When to use each:

Use implicit regularization (gradient descent) when:

- You want to achieve zero training loss (interpolation)
- The overparameterized regime ($p \gg n$)
- Tuning hyperparameters is expensive
- Architecture/initialization are well-designed

Use explicit regularization when:

- You cannot afford interpolation (small data, noisy labels)
- Underparameterized regime ($p < n$)
- You have domain knowledge about desired complexity
- Robustness to adversarial examples is critical

Summary: The Implicit Regularization Landscape

Key Takeaway

Core insights about implicit regularization:

1. **Selection among interpolators:** With $p \gg n$, infinitely many solutions achieve zero training loss. Gradient descent selects among them via implicit bias.
2. **Path-dependent bias:** The implicit regularization depends on parameterization, initialization, and optimizer—not just the function class.
3. **Norm minimization:** For many parameterizations, GD converges to minimum-norm solutions (nuclear norm for matrix factorization, path norm for deep networks).
4. **Margin maximization:** For separable classification, GD converges to max-margin solutions, implicitly solving the SVM problem.
5. **Architecture as regularization:** Different parameterizations of the same function class have different implicit biases (deep factorizations $\rightarrow$ stronger regularization).
6. **Optimization matters:** Batch size, learning rate, and optimizer choice significantly affect implicit bias and generalization.
7. **Implicit $\neq$ explicit:** Implicit regularization allows interpolation while still generalizing; explicit regularization prevents perfect fit.

The unified picture:

Modern deep learning's success comes from the **interaction** of:

- Overparameterization (creates many interpolating solutions)
- Implicit bias (selects “simple” solutions via optimization dynamics)
- Architecture design (shapes the notion of “simplicity”)
- Feature learning (adapts representations during training)

Classical learning theory focused on limiting capacity to prevent overfitting. Modern theory reveals that in overparameterized models, **how** we optimize matters as much as **what** we optimize.

Looking Ahead

Looking ahead:

We've seen that gradient descent has implicit biases toward simple solutions. But we haven't quantified this precisely:

- How do we measure the “effective capacity” of an overparameterized model?
- Can we derive generalization bounds that account for implicit regularization?
- What happens at the interpolation threshold where implicit regularization is weakest?

The next section on double descent addresses these questions, revealing a surprising phenomenon: test error can **decrease** beyond the interpolation threshold despite perfect training fit.

Double Descent and Modern Phenomena

We now examine a surprising modern discovery that challenges classical learning theory.

Opening Challenge

Classical theory predicts U-shaped test error (the bias-variance trade-off from Week 4-5). Modern deep learning shows **double descent** (Belkin et al. 2019): error decreases (first descent), then increases sharply at the interpolation threshold ($p \approx n$), then decreases again (second descent)!

Connection

Connection to Week 4-5 (Bias-Variance Trade-off): Classical trade-off predicts:

- Low complexity: high bias (model too simple to capture patterns), low variance (stable across different training sets)
- High complexity: low bias (model flexible enough to fit patterns), high variance (sensitive to training data randomness)
- Optimal complexity: balance between bias and variance (minimizes total expected error)

Double descent extends this: **very** high complexity can reduce variance again via implicit regularization! Beyond the interpolation threshold, having more parameters gives the optimizer more choices, allowing it to find smoother solutions that happen to fit the data.

Definition: Three Regimes of Learning

As the number of parameters p increases relative to the number of training samples n :

1. **Underparameterized** ($p < n$): Cannot fit all training data perfectly; classical bias-variance trade-off applies
2. **Interpolation threshold** ($p \approx n$): Just enough parameters to fit training data perfectly; peak test error occurs here (worst generalization!)
3. **Overparameterized** ($p \gg n$): Far more parameters than needed to fit data; test error decreases with more parameters due to implicit regularization!

Example: Double Descent in Practice

Train networks of varying width on MNIST:

- Width 1-10: High test error (underparameterized)
- Width 20: Peak test error (interpolation threshold)
- Width 20-100: Test error decreases (overparameterized)

Two descents: before and after interpolation!

Mental Model: Understanding Double Descent

Classical regime (underparameterized):

- Model cannot fit all data points
- Increasing complexity: bias $\downarrow$, variance $\uparrow$
- Minimum test error at optimal complexity

Interpolation threshold:

- Model barely fits all data points (one solution or very few)
- Forced to fit noise exactly
- No room for implicit regularization
- Peak test error (worst case!)

Overparameterized regime:

- Many solutions fit training data perfectly
- Gradient descent can choose among them via implicit bias
- Finds “simplest” interpolating solution
- Test error decreases with more parameters!

Key insight: Having more parameters gives optimizer more choices $\rightarrow$ can find better (simpler) solutions that happen to fit the data.

Analogy: Imagine connecting n points with curves:

- Degree $< n$: cannot pass through all points (underfit)
- Degree $= n$: unique polynomial, very wiggly (forced interpolation)
- Degree $\gg n$: many polynomials work, can choose smoothest one (overparameterized)

Author's Take

Informal double-descent explanation:

Test error can be viewed as approximation error plus variance from fitting finite/noisy data.

- At threshold: the model may be forced to fit noise (high variance)
- Beyond threshold: the optimizer may have many interpolating solutions and find a smoother one through implicit regularization

This pattern is observed and proved in selected models, but it is not guaranteed for every architecture, dataset, and noise distribution.

Theorem: Double Descent for Linear Models (Hastie et al. 2019)**Assume:**

- **Model:** Linear regression $y_i = x_i^T \theta^* + \varepsilon_i$ with $\theta^* \in \mathbb{R}^p$ and $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$
- **Data:** Features $x_i \sim \mathcal{N}(0, \Sigma)$ with covariance $\Sigma \in \mathbb{R}^{p \times p}$, eigenvalues $\lambda_1, \dots, \lambda_p$
- **Estimator:** Ridgeless least squares (minimum-norm interpolation)
 - If $p \leq n$: $\hat{\theta} = (X^T X)^{-1} X^T y$ (OLS, unique if full rank)
 - If $p > n$: $\hat{\theta} = X^T (X X^T)^{-1} y$ (minimum-norm solution)
- **Regime:** Vary p while holding n fixed

Then: The test risk $R(\hat{\theta}) = \mathbb{E}_{x,y} \left[(y - x^T \hat{\theta})^2 \right]$ exhibits double descent:

1. **Classical regime** ($p < n$): Risk decreases as p increases (more features $\rightarrow$ better fit)
2. **Interpolation peak** ($p = n$): Risk spikes (overfitting at threshold)
3. **Modern regime** ($p > n$): Risk decreases again as p increases (overparameterization helps!)

Risk formula (simplified):

$$R(\hat{\theta}) \approx \begin{cases} \sigma^2 \left(1 + \frac{p}{n}\right) & \text{if } p < n \\ \text{diverges} & \text{if } p = n \\ \sigma^2 \left(1 + \frac{n}{p}\right) & \text{if } p \gg n \end{cases} \quad (27)$$

The exact behavior depends on the eigenvalue spectrum of Σ .

Why does risk decrease for $p > n$?

Intuition: With many features ($p \gg n$), there are many interpolating solutions. Minimum-norm selection (implicit in ridgeless LS) chooses the smoothest, which generalizes better. This is implicit regularization from the optimization procedure.

 Key Takeaway

Double descent extends the classical bias-variance story in some overparameterized settings. Bigger models can generalize better due to implicit regularization beyond the interpolation threshold, but validation remains necessary because the pattern is not universal (it depends on the data distribution, model architecture, and training procedure).

Thought Experiment

Imagine fitting a polynomial to noisy data:

Underparameterized (degree 2 polynomial for 10 data points):

- Smooth parabola that cannot fit all points exactly
- Moderate test error (high bias because parabola is too simple, low variance because the fit is stable)

Interpolation threshold (degree 10 polynomial for 10 data points):

- Unique wiggly polynomial passes through all points, forced to fit noise exactly
- High test error (low bias because it fits training data perfectly, high variance because it's forced to fit noise in a specific wiggly way)

Overparameterized (degree 100 polynomial for 10 data points):

- Many polynomials can fit the data perfectly (infinitely many solutions)
- Gradient descent finds the smoothest one (minimum-norm interpolator among all degree-100 polynomials)
- Lower test error (low bias because it interpolates, moderate variance because smoothness constraint reduces sensitivity to noise)

Key insight: having **more** parameters (degree 100 vs degree 10) gives the optimizer **more choices**, allowing it to find better solutions (smooth interpolators rather than wiggly forced fits)!

Example: Double Descent Across Multiple Axes

Double descent has been observed across multiple dimensions:

Model size (width of neural network layers):

- Peak test error at width $\approx n$ (interpolation threshold)
- Test error decreases for width $\gg n$ (overparameterized regime)

Training epochs (number of passes through the training data):

- Initially: test error decreases (learning the signal from data)
- Middle: test error increases (starting to fit noise in the data)
- Late: test error decreases again (implicit regularization takes over, finding smoother solutions)

Sample size (varying n with fixed model complexity):

- Peak test error when $n \approx p$ (model complexity matches sample size)
- For fixed p , increasing n past the threshold reduces error (more data helps beyond the critical point)

✓ **Checkpoint****Double descent insights:**

1. Classical U-shaped curve is incomplete—there's a second descent!
2. Interpolation threshold ($p \approx n$) is the worst regime
3. Overparameterization ($p \gg n$) can improve generalization via implicit regularization
4. Pattern observed across models (linear, neural networks, trees) but not universal

Key question: When does double descent occur? When does it fail?

When Does Double Descent Occur?

Author's Take**Conditions favoring double descent (when we expect to see the phenomenon):**

1. **Interpolation regime:** Model must be able to fit training data perfectly (zero training loss achievable)
2. **Multiple solutions:** Many parameter settings achieve zero training loss (gives optimizer room to choose)
3. **Implicit regularization:** Optimization algorithm has bias toward “simple” solutions (e.g., gradient descent prefers low-norm solutions)
4. **Benign overfitting:** Fitting noise doesn't hurt too much in the overparameterized regime (noise is absorbed without affecting generalization much)

Conditions working against double descent (when the phenomenon may not appear):

1. **Explicit regularization:** Strong ℓ_2 penalty ($\lambda \|\theta\|_2^2$ added to loss) prevents interpolation (forces training error above zero)
2. **Early stopping:** Stopping training before interpolation avoids both the interpolation threshold peak and the second descent
3. **Strongly regularized tasks:** When explicit regularization dominates implicit bias (e.g., very large λ)
4. **Adversarial robustness requirements:** Interpolating solutions can be non-robust to adversarial perturbations (small worst-case input changes that fool the model)

Wrong Path: Always Use Maximally Overparameterized Models

Tempting conclusion: If bigger is better in overparameterized regime, always use huge models!

Why this is wrong:

1. **Computational cost:** Training scales with parameters
2. **Not always better:** Double descent isn't universal
3. **Diminishing returns:** Gains plateau eventually
4. **Robustness concerns:** Interpolating solutions can be non-robust
5. **Implicit regularization can fail:** Not all overparameterized models generalize

Better approach: Use validation to find appropriate model size. Overparameterization helps but isn't a panacea.

Connection

Course synthesis: Week 11 integrates concepts from all previous weeks:

- **Mathematical foundations (Week 1):** Convexity, Hessian matrices, concentration inequalities
- **PAC learning (Weeks 2-3):** Sample complexity, approximation-estimation trade-off
- **Model selection (Weeks 4-5):** Bias-variance trade-off extended to double descent
- **Kernel methods (Weeks 6-8):** NTK connects networks to kernel methods
- **Optimization (Weeks 9-10):** Gradient descent convergence, online learning

Central question answered: Why do overparameterized networks generalize?

- Overparameterization helps optimization (creates benign loss landscapes with few bad local minima)
- Implicit regularization prevents overfitting (gradient descent has bias toward simple solutions)
- Double descent: beyond interpolation threshold, more parameters help via implicit regularization!

Connection

Preview of Week 12: We've established:

- Networks can represent functions (expressivity)
- Networks can be trained (optimization)
- Networks have implicit biases (regularization)

Week 12 addresses remaining questions:

- How to quantify implicit regularization? (PAC-Bayes, compression)
- What are modern phenomena? (Grokking, scaling laws, emergence)
- When does theory match practice? (Realistic models)
- How to design better architectures? (Practical implications)

The Big Picture: Three Puzzles Resolved

Backbone resolution: Having explored expressivity, optimization, kernel theory, implicit regularization, and double descent, we can now directly answer our central question: **Why do overparameterized neural networks generalize?** The answer synthesizes everything we've learned this week.

Mental Model: Resolution of the Three Puzzles

Recall the three puzzles from the introduction. Here's how this week's theory resolves them:

Puzzle 1: Why doesn't gradient descent get stuck in local minima?

- Resolution: Overparameterization creates benign landscapes (loss surfaces that are relatively smooth and easy to optimize)
- Most critical points are saddles, not local minima (in high dimensions, having all Hessian eigenvalues positive is rare)
- SGD noise helps escape saddles (stochastic gradient descent's randomness pushes parameters away from saddle points)
- Very wide networks satisfy PL condition $\rightarrow$ exponential convergence (loss decreases exponentially fast despite non-convexity)

Puzzle 2: Why don't overparameterized networks overfit?

- Resolution: Implicit regularization from gradient descent (the optimization algorithm itself prefers certain solutions)
- Bias toward low-norm, low-rank, or max-margin solutions (gradient descent finds "simple" solutions among many possibilities)
- Effective capacity much smaller than parameter count (not all parameters are used equally)
- Optimization algorithm restricts hypothesis class (gradient descent explores only a small subspace of all possible parameter settings)

Puzzle 3: Why does interpolation (zero training error) not hurt?

- Resolution: Many interpolating solutions exist; gradient descent finds good ones (among all perfect-fit solutions, it chooses smooth ones)
- Implicit bias toward "simple" interpolators (low-norm or max-margin solutions that happen to fit the data)
- Double descent: beyond threshold, more parameters help (gives optimizer more choices to find better solutions)
- Benign overfitting in overparameterized regime (fitting noise doesn't significantly hurt test performance)

The unified story: Overparameterization helps both optimization (easier to find good solutions) AND generalization (implicit bias finds solutions that generalize well)!

Practice Problems

Practice Problems

Conceptual Understanding (★)

1. Why doesn't universal approximation guarantee practical success?

Hint: Think about what the theorem provides (existence) vs what we need (construction, sample complexity, computation).

2. Explain why deep networks are exponentially more efficient than shallow networks for compositional functions.

Hint: Consider how many features shallow networks need to enumerate vs how deep networks build hierarchically.

3. What's the difference between local minima and saddle points? Why aren't saddles problematic in high dimensions?

Hint: Count constraints: to be a local minimum, ALL Hessian eigenvalues must be positive. In high dimension, this is rare.

4. Distinguish explicit regularization (weight decay) from implicit regularization (gradient descent bias).

Hint: Explicit: added to objective function. Implicit: emergent property of optimization algorithm.

5. In the NTK regime, why does training become linear?

Hint: Features stay constant at initialization, only coefficients change—exactly like kernel methods!

6. Explain double descent in one sentence.

Hint: Test error peaks at interpolation threshold, then decreases again in overparameterized regime.

Theoretical Analysis (★★)

7. Prove ReLU networks can approximate any continuous piecewise linear function exactly.

Hint: Each ReLU creates one “bend”. Show how to combine them to match all breakpoints.

8. For two-layer linear network $f(x) = W_2 W_1 x$, show all critical points with $W_2 W_1 = W^*$ are global minima.

Hint: Loss is convex in product $W_2 W_1$. Any factorization with correct product has same loss.

9. Compute the NTK for a two-layer ReLU network: $f(x) = \left(\frac{1}{\sqrt{m}}\right) \sum_{i=1}^m a_i \max(0, w_i^T x)$.

Hint: Take gradient w.r.t. parameters, compute inner product. Use law of large numbers as $m \rightarrow \infty$.

10. Prove exponential convergence under the PL condition.

Hint: Use smoothness to get descent lemma, then apply PL to bound gradient norm by suboptimality.

11. Show that linear regression satisfies the PL condition when X has full rank.

Hint: Express gradient in terms of residual, use smallest eigenvalue of $X^T X$.

12. Verify that the NTK training dynamics are linear: $\frac{d\mathbf{f}_t}{dt} = -\frac{1}{n}\Theta(\mathbf{f}_t - \mathbf{y})$.

Hint: Chain rule: $\frac{df_t(x)}{dt} = \nabla_{\theta} f(x; \theta_t)^T \frac{d\theta_t}{dt}$. Use gradient flow for parameters.

Advanced Theory (★★★)

13. Prove Barron's theorem: functions with bounded Fourier spectrum achieve $O\left(\frac{1}{\sqrt{n}}\right)$ approximation error.

Hint: Use random feature approximation: sample projections according to Fourier spectrum, get Monte Carlo error rate.

14. Show the NTK remains constant during training in the infinite-width limit.

Hint: Each neuron's contribution scales as $\frac{1}{\sqrt{m}}$, so parameters move $O\left(\frac{1}{\sqrt{m}}\right)$. Gradient norm is $O(1)$, so NTK change is $O\left(\frac{1}{\sqrt{m}}\right)$.

15. Prove gradient descent on logistic loss converges in direction to the max-margin solution.

Hint: Show $\|w_t\| \rightarrow \infty$ but $\frac{w_t}{\|w_t\|}$ converges. Analyze limiting direction as minimizer of norm subject to correct classification.

16. Prove that deep linear networks with squared loss have no bad local minima under appropriate rank assumptions.

Hint: Show loss is convex in end-to-end mapping. Use Morse theory or direct analysis of critical points.

Computational Experiments (🤔)

17. Visualize loss landscapes for networks of varying width. Compare narrow vs. wide.

Hint: Fix two random directions in parameter space, plot loss as function of (α, β) . Observe: wider $\rightarrow$ smoother.

18. Train networks of varying width on a small dataset. Plot test error vs. parameters. Observe double descent?

Hint: Try MNIST subset with $n = 100$ samples. Vary width from 5 to 1000. Plot train/test curves.

19. Train a two-layer linear network. Does gradient descent find minimum nuclear norm or minimum Euclidean norm?

Hint: Initialize small. Train to zero loss. Measure nuclear norm $\|W_2 W_1\|_$ vs Frobenius norm $\|W_2\|_F^2 + \|W_1\|_F^2$. Compare to explicit nuclear norm minimization.*

20. Track margin during training on separable data. Does it increase monotonically? Compare to SVM.

Hint: Train logistic regression on separable 2D data. Plot margin $\min_i y_i \frac{w_i^T x_i}{\|w_i\|}$ vs iteration. Compare final direction to SVM solution.

21. Implement a very wide network and verify lazy training regime (compare to kernel ridge regression with NTK).

Hint: Train width-10000 network with small learning rate. Compute NTK at initialization. Compare predictions to kernel ridge regression with same kernel.

22. Visualize feature learning vs lazy training. Compare moderate width to very large width.

Hint: Track $\frac{\|\theta_t - \theta_0\|}{\|\theta_0\|}$ during training. Lazy regime: stays small. Feature learning: grows significantly.

Synthesis and Connections (🤔)

23. Compare NTK to classical kernel methods (Week 11 Background Section). Advantages and disadvantages?

Hint: NTK advantages: derived from architecture, adapts to task (at initialization). Disadvantages: lazy training limits representation, fixed after initialization. Classical kernels: designed for task, but no representation learning.

24. How does double descent relate to bias-variance trade-off (Week 4-5)? Contradiction or extension?

Hint: Extension! Classical: one descent. Modern: two descents with peak at interpolation threshold. Implicit regularization enables second descent.

25. Compare gradient descent convergence in: (a) convex optimization, (b) non-convex with PL, (c) overparameterized networks.

Hint: (a) Convex + smooth: $O(\frac{1}{T})$ to first-order stationary point; strongly convex: exponential. (b) PL: exponential convergence despite non-convexity. (c) Very wide: PL-like behavior via NTK; moderate width: empirically fast but less theory.

26. Design an experiment to test whether SGD and Adam have different implicit biases.

Hint: Train same architecture with SGD vs Adam on separable data. Compare learned solutions: margins, norms, flatness. Which finds “simpler” solution?

27. How does implicit regularization connect to PAC learning theory (Week 2-3)?

Hint: PAC: sample complexity depends on hypothesis class complexity. Implicit regularization: optimization restricts effective hypothesis class, reducing effective complexity below $O(p)$.

28. Why does overparameterization help both optimization AND generalization? Isn't this contradictory?

Hint: Not contradictory! More parameters $\rightarrow$ easier optimization (more degrees of freedom to avoid bad minima). Implicit bias $\rightarrow$ effective capacity stays controlled. Having many parameters $\neq$ using all of them.

29. Connect depth separation (expressivity) to sample complexity (learnability).

Hint: Approximation-estimation trade-off: better approximation (deep networks) $\rightarrow$ smaller approximation error $\rightarrow$ better sample complexity. But must also control estimation error via implicit regularization.

30. Critique: "NTK theory proves deep learning works." What's missing?

Hint: NTK explains: optimization is easy (lazy regime). NTK misses: (1) feature learning (finite width), (2) why learned features generalize, (3) architecture design, (4) practical training dynamics. NTK is one piece, not the full story!

✓ Checkpoint

Problem-solving strategies:

1. Conceptual: build intuition from mental models, connect to previous weeks
2. Theoretical: use proof templates (PL condition, loss landscape, NTK dynamics)
3. Computational: implement small-scale experiments to verify theory
4. Synthesis: connect concepts across weeks, identify gaps between theory and practice

These problems prepare you for Week 12's deeper dive into generalization theory and modern phenomena.

Summary

Key Takeaway

Week 11 main insights:

1. **Expressivity (Week 10):** Universal approximation guarantees existence of solutions; depth separation shows compositional efficiency. This week addresses: how does optimization find these solutions, and why do they generalize?
2. **Optimization landscape:** Overparameterized networks often have more benign landscapes under suitable assumptions, with fewer bad local minima in many models. High-dimensional parameter spaces make it hard for critical points to be local minima rather than saddles.
3. **Neural Tangent Kernel:** Infinite-width networks behave like kernel methods with a specific kernel determined by architecture and initialization, explaining easy optimization in the lazy training regime (where features stay frozen at initialization).
4. **Implicit regularization:** Gradient descent can prefer low-norm, low-rank, or max-margin solutions, aiding generalization in model-dependent ways. This emerges from the optimization dynamics, not from explicit penalty terms in the loss.
5. **Double descent:** Beyond the interpolation threshold ($p \approx n$), bigger models generalize better via implicit regularization. This extends the classical U-shaped bias-variance curve with a second descent.

 Mental Model: The Complete Week 11 Story

The central question: Why do overparameterized neural networks generalize?

Part 1: Expressivity (What can networks represent?)

- Universal approximation: any continuous function (existential result)
- Depth vs width: deep networks exponentially more efficient for compositional functions
- Connection to sample complexity via approximation-estimation trade-off

Part 2: Optimization (How do we train networks?)

- Non-convex optimization is hard in general
- Overparameterization creates benign loss landscapes
- High dimension helps: most critical points are saddles
- PL condition gives exponential convergence without convexity

Part 3: NTK regime (When are networks like kernels?)

- Infinite width $\rightarrow$ lazy training $\rightarrow$ kernel methods
- Training dynamics become linear
- Explains optimization but misses feature learning
- Practical networks (finite width) learn features

Part 4: Implicit regularization (Why don't networks overfit?)

- Gradient descent + loss + initialization = implicit bias
- Linear networks: low nuclear norm (low rank)
- Separable classification: max margin (like SVM)
- Deep networks: flat minima, “simple” solutions

Part 5: Double descent (Modern phenomena)

- Classical bias-variance: one descent
- Modern: second descent beyond interpolation threshold
- Overparameterization helps via implicit regularization
- Pattern observed but not universal

The unified answer: Overparameterization helps both optimization (benign landscape) AND generalization (implicit regularization). The optimization algorithm restricts the effective hypothesis class, preventing overfitting despite many parameters.

Author's Take

Week 11 answers: “Can we train neural networks?” Yes—overparameterization makes optimization tractable even though the problem is non-convex.

Week 12 will answer: “Why do trained networks generalize?” We’ll explore implicit regularization more deeply, compression bounds, margin theory, and modern phenomena (grokking, scaling laws, emergence).

What we’ve learned:

- Theory is catching up to practice (deep learning worked before we understood why)
- Overparameterization is a feature, not a bug (more parameters help both optimization and generalization)
- Implicit regularization is crucial (the optimization algorithm restricts what the model learns)
- Classical theory (VC dimension, PAC bounds) is incomplete for deep learning (these bounds are too pessimistic)

What remains mysterious:

- Exact characterization of implicit bias for deep nonlinear networks (we understand linear networks better)
- When does implicit regularization succeed vs fail? (what properties of data/architecture matter?)
- How to design architectures for better implicit bias? (can we engineer networks to have better inductive biases?)
- Connection between optimization and generalization (why does SGD generalize better than Adam or full-batch gradient descent?)

These questions motivate Week 12’s deeper exploration of generalization theory.

Connection

Preparation for Week 12:

Week 11 established the foundations:

- Networks can represent functions (expressivity)
- Networks can be optimized (landscape, PL condition, NTK)
- Networks have implicit biases (regularization)

Week 12 will build on these foundations:

- Quantifying implicit regularization (PAC-Bayes, compression)
- Modern phenomena (grokking, scaling laws, emergence, cata)
- Practical implications (architecture design, optimization cho
- Open problems and future directions

Key concepts to remember:

1. Overparameterization helps optimization and generalization
2. Implicit regularization explains why $p \gg n$ doesn't cause overfitting
3. NTK connects networks to kernels (but misses feature learning)
4. Double descent extends bias-variance trade-off
5. Classical theory incomplete—need new frameworks

These concepts form the backbone for understanding modern deep learning theory!

Historical note: The theory-practice gap in deep learning has narrowed significantly since 2012:

- 2012: AlexNet wins ImageNet (empirical success)
- 2016: Depth separation theorems (expressivity)
- 2018: NTK theory (optimization)
- 2018: Implicit margin maximization (generalization)
- 2019: Double descent (modern phenomena)
- 2020-present: Scaling laws, grokking, emergence

Theory evolves to explain practice, which guides new theory. Week 12 covers the latest developments!